LABORATÓRIO PRÁTICO

DOM + JavaScript

Exercícios de evolução a partir dos exemplos estudados em aula

Desenvolvimento Front End

IDEIA CENTRAL
Você receberá uma estrutura one-page pronta para organizar a atividade e poderá consultar os arquivos de exemplo estudados em aula. O objetivo, porém, não é reproduzir os exemplos. Cada atividade pede uma alteração de quantidade, comportamento, regra ou estrutura para que você pratique a leitura, a adaptação e a integração do código.

Como realizar a atividade

REGRA PRINCIPAL
A estrutura-base e os arquivos de exemplo podem e devem ser consultados. Entretanto, a simples cópia de um dos exemplos não atende ao exercício. Desenvolva cada solução na seção correspondente da página one-page, criando e adaptando seus próprios IDs, classes, variáveis, funções e conteúdos.

- Abra primeiro o arquivo index.html da estrutura one-page e localize a seção correspondente ao exercício que será realizado.
- Consulte os arquivos de exemplo relacionados ao exercício e identifique quais partes do HTML, CSS e JavaScript podem servir como referência.
- Desenvolva sua solução dentro da área reservada ao exercício, alterando uma etapa por vez e testando no navegador após cada mudança.
- Use o DevTools para experimentar alterações temporárias antes de decidir se elas devem entrar no arquivo definitivo.
- Sempre verifique se os IDs e classes usados no JavaScript correspondem aos elementos corretos do HTML e evite repetir IDs na mesma página.
- Ao terminar cada exercício, responda às perguntas de compreensão. O objetivo é conseguir explicar o código, não apenas fazê-lo funcionar.

Roteiro dos exercícios

Ex.	Componente	Principais recursos	Complexidade
1	Troca de cores e tema	style, classList, eventos	Inicial
2	Formulário semântico	form, fieldset, label, inputs	Inicial

Ex.	Componente	Principais recursos	Complexidade
3	Cadastro com validação	value, trim, if/else, reset	Inicial
4	Sistema com cinco abas	funções, IDs, classes, display	Intermediário
5	FAQ com accordion	estado, display, classes, eventos	Intermediário
6	Janela modal	parâmetros, conteúdo dinâmico, display	Intermediário
7	Carrossel	arrays, índice, src, disabled	Intermediário
8	Lista de tarefas 3.0	createElement, appendChild, remove, contador	Intermediário
EX. 1		Troca de cores e tema Nível: Inicial | Foco: manipulação de estilo e classes	

ARQUIVOS DE CONSULTA

trocacor(1).html · trocacor2(1).html

Objetivo

- Criar uma página em que o usuário possa alterar a aparência visual por meio de botões, utilizando o DOM.

O que sua versão deve ter

☐ Um título e um pequeno texto explicativo.
☐ Quatro botões de cores fixas escolhidas por você.
☐ Um botão chamado “Cor original” que restaure a aparência inicial.
☐ Um botão separado para alternar entre modo claro e modo escuro.
☐ O texto do botão de tema deve indicar qual ação ocorrerá no próximo clique.

Passo a passo

1 Crie a estrutura HTML com o título, o texto e os botões. Dê um ID ao botão de tema para que ele possa ser localizado no JavaScript.

2 No CSS, defina a aparência inicial da página e crie uma classe para representar o modo escuro.

3 No JavaScript, faça os botões de cor alterarem o background da página. Você pode consultar o exemplo de troca de cor para lembrar como acessar document.body.style.

4 Implemente o botão “Cor original” fazendo a página retornar à cor inicial definida por você.

5 Implemente o botão de tema usando classList para adicionar ou remover a classe do modo escuro.

6 Depois de alternar o tema, atualize o texto do botão conforme o estado atual.

7 Teste várias combinações: mude a cor, ative o modo escuro, volte ao modo claro e restaure a cor original.

Checklist

☐ Todos os botões respondem ao clique.
☐ O modo escuro pode ser ligado e desligado.
☐ O texto do botão de tema muda corretamente.
☐ A página volta ao estado inicial sem recarregar.

Explique o que você fez

• Qual diferença existe entre alterar uma propriedade com style e ativar uma classe com classList?
• Qual elemento do HTML o JavaScript precisa localizar para alterar o texto do botão de tema?

EX. 2	Formulário semântico Nível: Inicial | Foco: estrutura de formulário, associação de campos e semântica

ARQUIVOS DE CONSULTA

form(1).html · style(6).css

Objetivo

• Construir um formulário completo e semanticamente organizado antes de aplicar validações ou manipulações com JavaScript.

Contexto sugerido

• Crie uma ficha de inscrição para um curso, evento, processo seletivo ou atividade acadêmica. O conteúdo pode ser personalizado, mas a estrutura solicitada deve ser mantida.

Elementos obrigatórios

☐ Um elemento form envolvendo todos os campos.
☐ Pelo menos dois fieldsets com legend para organizar grupos de informações.
☐ Campos para nome e e-mail, com labels corretamente associados.
☐ Pelo menos um campo de outro tipo, como date ou password.
☐ Um grupo de radio buttons para uma escolha única.
☐ Um grupo de checkboxes que permita múltiplas escolhas.
☐ Um select com pelo menos quatro opções.
☐ Um textarea para uma resposta mais longa.
☐ Botões para enviar e limpar o formulário.

Passo a passo

1. Crie a tag form e escolha um tema para a ficha de inscrição. Mantenha todos os controles relacionados dentro dessa estrutura.

2. Separe as informações em grupos usando fieldset e legend. Pense em grupos como “Dados pessoais”, “Preferências” e “Finalização”.

3. Crie os campos de nome e e-mail. Para cada campo, associe o atributo for do label ao id do input correspondente.

4. Adicione pelo menos um campo de outro tipo, como date ou password, e escolha quais campos devem ser obrigatórios usando required.

5. Monte um grupo de radio buttons. Todos os radios do mesmo grupo devem compartilhar o mesmo atributo name para permitir apenas uma escolha.

6. Adicione checkboxes para opções que possam ser escolhidas simultaneamente. Use IDs diferentes para cada opção.

7. Crie um select com pelo menos quatro option e inclua também um textarea para uma resposta mais longa.

8. Adicione um botão type="submit" e um botão type="reset". Teste a limpeza dos campos e a validação nativa dos campos obrigatórios.

9. No CSS, organize os campos de forma legível. Você pode usar classes para grupos verticais e grupos em linha, tomando o exemplo fornecido apenas como referência.

Checklist

☐ Cada label está associado ao campo correto.

☐ Não existem IDs repetidos.

☐ Os radio buttons de escolha única compartilham o mesmo name.

☐ Os checkboxes permitem selecionar mais de uma opção.

☐ O select e o textarea aparecem corretamente na página.

☐ Os botões de envio e limpeza têm tipos adequados.

☐ Os campos obrigatórios são identificados e testados no navegador.

Explique o que você fez

- Qual é a relação entre o atributo for de um label e o id de um input?
- Qual é a diferença entre id e name em um campo de formulário?
- Por que os radio buttons do mesmo grupo precisam compartilhar o mesmo name?
- Para que servem fieldset e legend em um formulário?

EX. 3	Cadastro com validação Nível: Inicial | Foco: leitura de campos, condições e mensagens

ARQUIVOS DE CONSULTA

validacaodeformulario(1).html · validacaodeformulario2(1).html · form(1).html · style(6).css

Objetivo

- Construir um formulário de cadastro e aplicar regras de validação com JavaScript, utilizando a estrutura de campos estudada no exercício anterior.

Campos obrigatórios

☐ Nome
☐ Idade
☐ E-mail
☐ Senha
☐ Confirmação da senha

Regras de validação

☐ O nome não pode ficar vazio.
☐ A idade deve ser igual ou superior a 16 anos.
☐ O e-mail não pode ficar vazio e deve usar input type="email".
☐ A senha deve ter no mínimo 6 caracteres.
☐ Senha e confirmação devem ser iguais.
☐ Se houver erro, exiba uma mensagem em vermelho.
☐ Se tudo estiver correto, exiba uma mensagem em verde e limpe o formulário.

Passo a passo

1. Monte o formulário usando label ligado ao input correspondente por meio de for e id.
2. Crie um elemento separado para mostrar as mensagens de erro ou sucesso e dê a ele um ID.
3. No JavaScript, crie uma função de validação e leia o valor de cada campo.
4. Use trim() no nome para evitar que apenas espaços sejam considerados um nome válido.
5. Converta a idade para número antes de compará-la.

6 Organize as verificações em uma sequência de if / else if / else. Pense na ordem em que os erros devem ser testados.

7 Na situação de sucesso, altere a mensagem e sua cor e depois use reset() para limpar os campos.

8 Teste propositalmente cada erro individualmente antes de testar um cadastro válido.

Testes mínimos

☐ Nome vazio
☐ Idade 15
☐ Senha com menos de 6 caracteres
☐ Senhas diferentes
☐ Cadastro totalmente válido

Explique o que você fez

- Por que a ordem dos if / else if influencia a mensagem mostrada ao usuário?
- O que .value retorna quando você lê um input?
- Para que serve reset() neste exercício?

EX. 4

Sistema com cinco abas
Nível: Intermediário | Foco: IDs relacionados, funções e classes

ARQUIVOS DE CONSULTA

abasdeconteudo.html · abasdeconteudo2.html

Objetivo

- Expandir o exemplo de navegação por abas para uma interface maior, mantendo apenas um conteúdo visível por vez.

Sua interface deve ter cinco abas

☐ Início
☐ Cursos
☐ Professores
☐ Projetos
☐ Contato

Requisitos obrigatórios

☐ Cinco botões e cinco blocos de conteúdo correspondentes.
☐ Somente uma área de conteúdo pode ficar visível por vez.
☐ O botão da aba ativa deve receber uma aparência diferente.
☐ A primeira aba deve aparecer selecionada ao carregar a página.
☐ Você deve aproveitar um padrão de IDs que permita relacionar botão e conteúdo.

Passo a passo

1 Estude como os exemplos relacionam nomes como aba1, aba2 e btnAba1, btnAba2. Observe que o número permite localizar elementos semelhantes.

2 Crie os cinco botões e os cinco conteúdos no HTML. Evite IDs repetidos.

3 No CSS, defina uma classe base para o conteúdo escondido e outra aparência para o botão ativo.

4 Crie uma função responsável por esconder todos os conteúdos e retirar a classe ativa de todos os botões.

5 Crie a função que recebe qual aba foi escolhida e chama primeiro a função de limpeza.

6 Depois, faça apenas o conteúdo correspondente ficar visível e destaque o botão correspondente.

7 Teste clicando rapidamente em todas as abas e confirme que nunca ficam duas abertas ao mesmo tempo.

Explique o que você fez

- Como o valor recebido pela função pode ser usado para montar o seletor de um ID?
- Por que é útil ter uma função separada apenas para “resetar” todas as abas?

EX. 5	FAQ com accordion Nível: Intermediário | Foco: controle de estado e abertura/fechamento

ARQUIVOS DE CONSULTA

menuaccordion(1).html · menuaccordion2(1).html · accordion3.html

Objetivo

- Criar um FAQ com quatro perguntas em formato sanfona, abrindo somente a resposta selecionada.

Perguntas sugeridas

☐ O que é HTML?
☐ O que é CSS?
☐ O que é JavaScript?
☐ O que é DOM?

Comportamento esperado

☐ Todas as respostas começam fechadas.
☐ Ao clicar em uma pergunta, sua resposta aparece.
☐ Ao abrir uma pergunta, qualquer outra resposta aberta deve ser fechada.
☐ Ao clicar novamente na pergunta que já está aberta, ela deve fechar.
☐ O indicador visual deve mudar de ▼ para ▲ quando o item estiver aberto.

Passo a passo

1 Monte quatro pares de botão + painel e crie uma forma consistente de relacionar cada botão ao seu conteúdo.

2 No CSS, deixe os painéis escondidos no estado inicial.

3 Crie uma função que consiga fechar todos os painéis e restaurar os textos ou ícones dos botões.

4 Na função acionada pelo clique, descubra qual painel foi escolhido e verifique se ele já estava aberto.

5 Feche tudo antes de decidir se o item clicado deve ser aberto novamente.

6 Se ele estava fechado, mostre o painel e altere o indicador do botão.

7 Teste o comportamento alternando entre perguntas e clicando duas vezes na mesma pergunta.

DESAFIO OPCIONAL

Se sua versão básica estiver funcionando, tente reescrever o accordion usando querySelectorAll(), forEach(), addEventListener() e classList, como no exemplo mais elaborado.

Explique o que você fez

- Que informação você precisa guardar ou verificar para saber se um painel já estava aberto?
- Qual é a vantagem de fechar todos os painéis antes de abrir o selecionado?

EX. 6	Janela modal com conteúdos diferentes Nível: Intermediário | Foco: funções com parâmetros e conteúdo dinâmico

ARQUIVOS DE CONSULTA

janelamodal(1).html · janelamodal2(1).html

Objetivo

- Criar uma mesma janela modal capaz de mostrar mensagens diferentes dependendo do botão clicado.

Sua página deve possuir

☐ Três botões: “Aviso”, “Ajuda” e “Privacidade”.
☐ Uma única estrutura de modal escondida inicialmente.
☐ Um título e um parágrafo dentro do modal que serão substituídos dinamicamente.
☐ Um botão “Fechar” e um X no canto superior para encerrar a janela.

Passo a passo

1 Crie os três botões da página e a estrutura única do modal.

2 Dê IDs ao título e ao texto do modal para que o JavaScript consiga substituí-los.

3 Crie uma função abrir() que receba pelo menos dois parâmetros: título e mensagem.

4 Cada botão deve chamar a mesma função, mas enviar conteúdos diferentes.

5 Dentro da função, atualize o título, atualize a mensagem e torne o fundo do modal visível.

6 Crie uma função fechar() para esconder o modal novamente.

7 Associe a função fechar() tanto ao botão interno quanto ao X.

8 Teste os três botões e confirme que a mesma janela mostra conteúdos diferentes.

Explique o que você fez

- Qual é a vantagem de usar uma única função com parâmetros em vez de criar três funções de abertura?
- Quais elementos precisam de ID para que título, texto e visibilidade possam ser alterados?

EX. 7

Carrossel com cinco imagens
Nível: Intermediário | Foco: arrays, índice e atualização do DOM

ARQUIVOS DE CONSULTA

carrossel.html · carrossel2.html

Objetivo

- Ampliar o carrossel estudado em aula e sincronizar imagem, legenda, posição e controles.

Requisitos

☐ Cinco imagens armazenadas em um array.
☐ Cinco legendas correspondentes armazenadas em outro array.
☐ Botões “Anterior” e “Próximo”.
☐ Indicador no formato “Imagem X de 5”.
☐ A legenda deve mudar sempre junto com a imagem.

Escolha um dos dois comportamentos

- Opção A: ao ultrapassar a última imagem, o carrossel volta para a primeira; ao voltar antes da primeira, ele vai para a última.
- Opção B: o carrossel não ultrapassa os limites e desabilita “Anterior” na primeira imagem e “Próximo” na última.

Passo a passo

1 Escolha cinco URLs de imagem ou cinco imagens locais e coloque-as em um array.

2 Crie um segundo array com cinco legendas, mantendo a mesma ordem das imagens.

3 Crie a variável que representa a posição atual, começando em 0.

4 Faça os botões chamarem uma função informando a direção do movimento.

5 Dentro da função, altere o índice e aplique a regra de limite escolhida por você.

6 Atualize o src da imagem usando o índice atual.

7 Atualize também a legenda e o indicador numérico.

8 Teste o primeiro e o último item com atenção para identificar erros de índice.

Explique o que você fez

- Por que o primeiro elemento do array usa índice 0, mas o indicador da interface deve mostrar “Imagem 1”?
- O que fotos.length representa e por que ele é útil?

EX. 8	Lista de tarefas 3.0 Nível: Intermediário | Foco: criação e remoção dinâmica de elementos

ARQUIVOS DE CONSULTA

todolist(1).html · todolist2(1).html

Objetivo

- Combinar recursos presentes nas duas versões de lista de tarefas em uma implementação mais completa.

Sua lista deve permitir

☐ Adicionar uma tarefa pelo botão.
☐ Impedir a inclusão de texto vazio.
☐ Adicionar também ao pressionar Enter.
☐ Exibir o total atual de tarefas.
☐ Excluir uma tarefa individualmente.
☐ Limpar todas as tarefas de uma vez.
☐ Limpar o campo e devolver o foco a ele após adicionar uma tarefa.

Passo a passo

1 Crie o campo de texto, os botões necessários, um contador e uma lista vazia no HTML.

2 No JavaScript, guarde em constantes as referências do campo, da lista e do contador.

3 Crie uma variável numérica para representar a quantidade de tarefas existentes.

4 Na função adicionar(), leia o valor do campo e use trim() antes de validar.

5 Se o texto for válido, crie um novo li com document.createElement().

6 Inclua o texto da tarefa e crie um pequeno botão “Excluir” para esse item.

7 Associe ao botão individual uma função que remova apenas o li correspondente e atualize o contador.

8 Insira o item na lista usando appendChild() ou método equivalente já estudado.

9 Atualize o contador, limpe o campo e use focus().

10 Adicione um evento de teclado ao campo para chamar adicionar() quando a tecla for Enter.

11 Implemente “Limpar tudo”, zerando tanto a lista quanto o contador.

12 Teste adicionar, excluir, limpar tudo e adicionar via teclado.

ATENÇÃO
Ao excluir uma tarefa, o contador precisa diminuir. Ao limpar tudo, ele precisa retornar a zero. Teste essas duas situações separadamente.

Explique o que você fez

- Por que createElement() é necessário quando o item ainda não existe no HTML inicial?
- Qual é a diferença entre remover um único elemento e limpar todo o conteúdo de uma lista?
- O que o evento keydown permite detectar?

Checklist final

Abra sua página no navegador e faça os testes abaixo. Um exercício que “parece pronto” pode ainda possuir erros de ligação entre HTML e JavaScript.

☐ Cada solução foi desenvolvida na seção correta da estrutura one-page.

☐ O menu de navegação e os links “Voltar ao topo” continuam funcionando após suas alterações.

☐ Não existem IDs duplicados na mesma página.

☐ Todos os IDs usados no JavaScript existem no HTML.

☐ Os nomes das classes manipuladas pelo JavaScript correspondem às classes definidas no CSS.

☐ Nenhum botão importante recarrega a página sem necessidade.

☐ As mensagens de erro aparecem nas situações previstas.
☐ Os componentes que começam escondidos realmente estão escondidos ao carregar a página.
☐ As ações de abrir/fechar ou mostrar/esconder funcionam repetidamente, não apenas uma vez.
☐ Os contadores e indicadores são atualizados depois de cada ação.
☐ O Console do navegador não apresenta erros em vermelho durante os testes.
☐ Você consegue explicar pelo menos três linhas importantes do JavaScript em cada exercício que utiliza script.

LEMBRETE
Consultar um exemplo é parte do aprendizado. O objetivo é descobrir quais peças do código você precisa reutilizar, quais precisa adaptar e por quê. Faça mudanças pequenas, teste com frequência e use o DevTools para investigar o que está acontecendo na página.